



EÖTVÖS LORÁND UNIVERSITY

FACULTY OF INFORMATICS

DEPT. OF ALGORITHMS AND THEIR APPLICATIONS

Terraforming Mars in OpenGL

Supervisor:

Csongor Csanád Karikó

Doctoral Student

Author:

Norbert Varga

Computer Science BSc

Budapest, 2025

Contents

1	Introduction	3
2	User documentation	4
2.1	Hardware and software requirements	4
2.1.1	Hardware	4
2.1.2	Software	4
2.2	Installation	4
2.3	Usage of the application	4
2.3.1	Menu	5
2.3.2	Game	5
2.3.3	Debug tools	9
3	Developer documentation	10
3.1	Developer environment	10
3.1.1	Utilized libraries	10
3.1.2	Compilation	11
3.1.3	Runtime environment	11
4	Conclusion	12
	Acknowledgements	13
A	Simulation results	14
	Bibliography	15
	List of Figures	15
	List of Tables	16
	List of Algorithms	17

List of Codes	18
----------------------	-----------

Chapter 1

Introduction

During my studies, I learned object-oriented planning and software development, and have made it my own. I have come to truly enjoy thinking about and designing the architecture of software. As a result, I was keen on writing my thesis about a project where I could utilize and sharpen these skills.

Another subject I really enjoyed was the computer graphics course. What I liked most about it is how it required me to combine the knowledge I gained from all the other courses I took at the university. From linear algebra to algorithms to analysis and more, computer graphics uses parts from almost every computer science related field. I already knew after I finished the course, that I probably want to write my thesis about this topic.

To combine both of these into one, I chose to implement the board game Terraforming Mars in C++ with OpenGL. This way I had to design the model for the game logic, but also manage the view and graphics rendering of the application. The program would let the user play the solo version of Terraforming Mars with 3D graphics.

Chapter 2

User documentation

2.1 Hardware and software requirements

2.1.1 Hardware

- 64-bit CPU with the x86 instruction set
- GPU supporting OpenGL 4.6
- 1 GB RAM
- mouse, optionally keyboard

2.1.2 Software

- Operating system: Windows 11
- GPU driver supporting OpenGL 4.6

2.2 Installation

The usage of the software does not require any form of setup or installation. After extracting the archive, the application can be run with the *view.exe* executable.

2.3 Usage of the application

The user can quit the application at any point by clicking the *Close* button of the window, or by pressing the following keyboard combination: **Alt + F4**

Fullscreen mode can be toggled at any point by pressing the following key: **F11**

2.3.1 Menu

The menu screen has the following three buttons the user can interact with:

- **Continue:** this button is only shown if a game has already been started. Pressing this button brings back the player to the game screen with the ongoing game.
- **New Game:** pressing this button creates a new game and transitions the app to the game screen. If there was an ongoing game, it is lost forever.
- **Quit:** pressing this button exits the application, as if the *above detailed* methods were used. If there was an ongoing game, it is lost forever.

2.3.2 Game

The player should be familiar with the *game rules*, in particular with the additions of the solo variant. Furthermore, these modifications are in effect:

- The player always controls the beginner corporation (10 cards for free and 42 Credits).
- Only about half of the cards from the basic set are included.

States

The game can be in different so called '*states*'. A state just means that certain actions may or may not be available to the player. The states are:

- **Action state:** This state is almost the same as the *Action phase* of the board game. The player can play cards from their hand, use standard projects or convert resources, use actions on their active cards and end the generation with the *End button*. The slight difference here is when using the *Sell patents* standard project, the game enters the *Sell state* where they can actually sell cards in their hand.
- **Sell state:** When using the *Sell patents* standard project, the game enters this state. In this state only the following actions can be performed:
 - All cards in the *hand* glow red. The player has the option to drag out any card from their hand and play it. Playing the card sells it, discarding the

card and giving 1 Credit to the player. For more detail about dragging out and playing cards, check the description of the *hand*.

- The label of the *End button* is replaced. Clicking on it exits the *Sell state*, going back to the *Action state*.
- **Payment confirmation state:** When playing a card or using an action that can be bought with resources, the game will enter this state. In this state every action is disabled, and a window is shown where the player can confirm the amount of the resources they wish to spend. Confirming the payment advances the game, and returns to the *Action state*.
- **Placement confirmation state:** When playing a card or using a standard project or conversion that places a tile, the game will enter this state. The border of the tiles on *Mars* that are valid will be colored white, and the others will be dark. In this state every action is disabled, but the tiles of *Mars* are clickable, and clicking on a valid one places the tile there. A hint text informs the player what kind of tile they are placing currently. Confirming the placement advances the game, and returns to the *Action state*.
- **Research state:** In the *Action state*, clicking on the *End button* automatically performs the production phase, and after that the game will enter this state. In this state four cards will appear on the screen. The player needs to decide how many and which they want to purchase. Initially all of them are set for purchasing, and the player can toggle them between purchasing and discarding. This is backed up with visual clues. A hint text informs the player how much they are going to play for the selected cards. The label of the *End button* is replaced. Clicking on it confirms the research, and the game returns to the *Action state*.
- **Post last generation state:** After ending the last generation and performing production phase, if the player has enough Plant resources to convert them to a greenery, the game will enter this state. Here, only the *Convert Plants to greenery* standard project can be used and the *End button* to finish the game.
- **Game over state:** After the last generation the game will enter this state. The game will display if the player has won or not, and the amount of victory points they have earned. No actions are available in this state.

Elements

The game screen consists of the following visual elements:

- **Menu button:** Located in the top left corner. This button can always be clicked, and takes the player back to the menu screen. The ongoing game gets paused, and can be resumed by selecting the *Continue button* on the menu screen.
- **End button:** Located on the right side. It can have multiple behaviors, depending on the *state*.
- **Standard projects and converting resources:** Located on the left side of the screen under the *Menu button*. When they can be used, hovering over them gives the visual clue of enlarging the row of the action. From top to bottom:
 - **Sell patents:** The application enters the *Sell state*.
 - **Power Plant:** For 11 Credits, you increase your Energy production 1 step.
 - **Asteroid:** For 14 Credits, you raise temperature one step.
 - **Aquifer:** For 18 Credits, you place 1 ocean tile.
 - **Greenery:** For 23 Credits, you place 1 greenery tile.
 - **City:** For 25 Credits, you place 1 city tile and increase your Credit production 1 step.
 - **Convert Plants to greenery:** For 8 Plant resources, you place 1 greenery tile.
 - **Convert Heat to temperature:** For 8 Heat resources, you raise temperature 1 step.
- **Generation counter:** Located in the bottom left corner. Displays the current generation.
- **Global parameters:** Located on the upper right side of the screen. Shown parameters in order are: temperature, ocean count, oxygen, terraform rating. The three global parameter's indicator turn green when they have reached their respective goal.

- **Resources:** Located on the lower right side of the screen. Consists of three columns: the first is the amount of resources, the second specifies the resource type and the third is the production output of said resource. The resources from top to bottom are as follows: Credit, Steel, Titanium, Plant, Energy, Heat. Values cannot be reduced below 0, except for Credit production which can be at a minimum of -5.

- **Hand:** Located at the bottom of the screen. Cards are by default hoverable: moving the mouse over a card enlarges it, showing the full card.

Some *states* allow the cards to be 'played'. In this case when a card is hovered, it can be dragged around with the mouse by pressing and holding the left mouse button. Two things can happen when the player releases the card: if the position of the mouse (and card) is above an invisible line, the card will get 'played', which is defined by the *state*. Otherwise, it will go back to the player's hand. There is always a visual indicator showing which one is about to happen.

- In the *Action state*, all cards that can be played will have a green glowing effect. If the mouse is over the invisible line, the card will have a blue glowing effect. The player can drag non-playable cards, but going over the invisible line automatically lets go of the dragged card.
- In the *Sell state*, all cards will have a red glowing effect. If the mouse is over the invisible line, the card will have a faded out effect. 'Playing' a card will sell it.

- **Played card panels:** Four panels are located on the screen: action cards on the bottom left, event cards, automated cards and effect cards on the top. The panels open when they get clicked on, and close when any part of their body gets clicked, except the cards. They can also be toggled with the following keyboard keys: **U**, **I**, **O** and **P**. These can be opened at most times, but they do get closed automatically at certain events.

By default, only 8 cards can fit on a panel. If there are more out of the view, an arrow will be shown, which can be used to step one page in that direction. This can also be done with the **Left** and **Right** keys.

The action panel has further functionality: the card's actions can be used from here. If a card can be used, it will have a blue glowing effect. In this

case, clicking on it will use the action of the card. If it was already used in the generation, it will have a faded out effect. If it was not used in the generation but can't be used, it won't have any effects.

- **Mars** Located in the center of the board. Empty tiles show their respective placement bonuses. Tiles owned by the player will have a player icon. Tiles reserved for oceans have a blue border. Placement of tiles is described at *Placement confirmation state*.

2.3.3 Debug tools

There are a few debug tools built in to the application. Both screens have a debug window that can be toggled by pressing **Ctrl + D**.

Menu

The debug window allows the user to set a seed for their game. Starting two games with the same seed means the random tiles will generate to the same place, and cards will come in the same sequence.

Game

The debug window allows the user to give themselves resources, resource production, raise their terraform rating, draw cards, raise temperature, raise oxygen and place ocean, greenery or city tiles.

Warning! The debug tool may cause crashes! I advise the user to only use it in the Action state.

Pressing **F3** toggles the basic camera and the editorial camera. With the editorial camera, the user can use look around with the mouse and fly around with the following keys: **A**, **W**, **S**, **D**, **E**, **Q**¹.

¹A - left, W - forward, S - back, D - right, E - up, Q - down

Chapter 3

Developer documentation

3.1 Developer environment

The program uses the **C++20** major version. Libraries were installed using the **vcpkg** package manager.

I developed the application on a laptop with **Windows 11**. I used Microsoft's IDE, **Visual Studio 2022** utilizing the **Desktop development with C++** workload and the **GLSL language integration** extension.

3.1.1 Utilized libraries

The following libraries and versions have been used for the application. Here is the command that was used to install the libraries using vcpkg:

```
.\vcpkg install sdl3 sdl3-image[jpeg,png] glew glm imgui[opengl3-binding,sdl3-binding] freetype --triplet x64-windows
```

- **OpenGL:** (Apache License 2.0) - graphics rendering library.
- **OpenGL Extension Wrangler library (GLEW):** (MIT License) - library providing an interface to OpenGL.
- **OpenGL Mathematics:** (MIT License) - math functions, vectors, matrices, transformations, easing functions and more.
- **Simple DirectMedia Layer (SDL):** (zlib License) - window and input manager.

- **Dear ImGui:** (MIT License) - graphical user interface for debugging and tooling.
- **FreeType:** (The FreeType Project License) - library for font rendering.

3.1.2 Compilation

The project needs to be compiled with the following macros being defined: `SDL_MAIN_HANDLED`, `GLM_ENABLE_EXPERIMENTAL`, `GLM_FORCE_SWIZZLE`.

3.1.3 Runtime environment

Compiling the program in *Release* mode assembles the program in the *x64/Release/* directory. *view.exe* expects the assets, fonts and shaders to be in their respective directories¹: Visual Studio 2022 should take care of this. This directory can now be moved, as this is the full application.

¹*assets/*, *fonts/*, *shaders/*

Chapter 4

Conclusion

d

Acknowledgements

Appendix A

Simulation results

List of Figures

List of Tables

List of Algorithms

List of Codes